Linux Commands Reference Guide

1. clear

Description: Clears all previous output from the terminal screen, giving a fresh, empty prompt. It does not delete any files or affect command history.

Syntax: `clear`

Output:

```
isha@Ubuntu:~/Desktop$ clear
```

Note: *(Terminal screen is wiped clean; only a fresh prompt is shown)*

2. pwd

Description: Prints the absolute path of the current working directory (Print Working Directory).

Syntax: `pwd`

Output:

```
isha@Ubuntu:~/Desktop$ pwd
/home/user/documents
```

3. ls

Description: Lists the files and directories present in the current or specified directory. Common options: -l (long/detailed listing), -a (show hidden files), -h (human-readable sizes).

Syntax: `ls [options] [directory]`

Output:

```
isha@Ubuntu:~/Desktop$ ls
Documents  Downloads  notes.txt  Pictures
```

4. mkdir

Description: Creates one or more new directories. The -p option creates parent directories as needed if they do not already exist.

Syntax: `mkdir [options] directory_name`

Output:

```
isha@Ubuntu:~/Desktop$ mkdir Projects
isha@Ubuntu:~/Desktop$ mkdir -p Projects/App/src
```

Note: *(No output on success; the directories Projects, App and src are created)*

5. cd

Description: Changes the current working directory to the specified directory. cd .. moves up one level; cd ~ or cd with no argument moves to the home directory.

Syntax: `cd [directory]`

Output:

```
isha@Ubuntu:~/Desktop$ cd Projects
```

```
isha@Ubuntu:~/Desktop$ pwd
/home/user/Projects
```

6. touch

Description: Creates a new empty file if it does not exist or updates the access and modification timestamp of an existing file without changing its content.

Syntax: `touch filename`

Output:

```
isha@Ubuntu:~/Desktop$ touch report.txt
isha@Ubuntu:~/Desktop$ ls -l report.txt
-rw-r--r-- 1 user user 0 Sep  3 11:20 report.txt
```

7. rm

Description: Removes (deletes) files or directories. -r removes directories recursively and -f forces deletion without confirmation prompts. Use with caution — deleted files are not sent to a recycle bin.

Syntax: `rm [options] file_or_directory`

Output:

```
isha@Ubuntu:~/Desktop$ rm report.txt
isha@Ubuntu:~/Desktop$ rm -rf OldProject/
```

Note: *(No output on success; report.txt and the OldProject directory are permanently deleted)*

8. rmdir

Description: Removes empty directories only. If the directory contains any files or subdirectories, the command fails with an error.

Syntax: `rmdir directory_name`

Output:

```
isha@Ubuntu:~/Desktop$ rmdir EmptyFolder
rmdir: failed to remove 'Projects': Directory not empty
```

Note: *(No output on success) (if the directory is not empty)*

9. mv

Description: Moves files or directories from one location to another and can also be used to rename a file or directory by moving it to a new name within the same location.

Syntax: `mv source destination`

Output:

```
isha@Ubuntu:~/Desktop$ mv notes.txt Documents/
isha@Ubuntu:~/Desktop$ mv oldname.txt newname.txt
```

Note: *(No output on success; notes.txt is moved into Documents/ and oldname.txt is renamed to newname.txt)*

10. cat

Description: Displays (concatenates and prints) the entire contents of one or more files to the terminal. Can also be used to combine multiple files.

Syntax: `cat filename`

Output:

```
isha@Ubuntu:~/Desktop$ cat notes.txt
Meeting at 10 AM
Bring laptop and charger
Submit report by Friday
```

11. cat -n

Description: Displays the contents of a file just like cat, but prefixes every output line with its line number, which is useful for referencing specific lines.

Syntax: `cat -n filename`

Output:

```
isha@Ubuntu:~/Desktop$ cat -n notes.txt
     1     Meeting at 10 AM
     2     Bring laptop and charger
     3     Submit report by Friday
```

12. cp

Description: Copies files or directories from a source location to a destination, leaving the original file intact. The -r option is required to copy directories recursively.

Syntax: `cp [options] source destination`

Output:

```
isha@Ubuntu:~/Desktop$ cp notes.txt notes_backup.txt
isha@Ubuntu:~/Desktop$ cp -r Documents/ Documents_Backup/
```

Note: *(No output on success; a copy of the file/directory is created at the destination)*

13. tac

Description: Displays the contents of a file in reverse order — the last line is printed first and the first line is printed last (tac is 'cat' spelled backwards).

Syntax: `tac filename`

Output:

```
isha@Ubuntu:~/Desktop$ tac notes.txt
Submit report by Friday
Bring laptop and charger
Meeting at 10 AM
```

14. head

Description: Displays the first part of a file, by default the first 10 lines. The -n option specifies a custom number of lines to display.

Syntax: `head [-n number] filename`

Output:

```
isha@Ubuntu:~/Desktop$ head -n 2 notes.txt
Meeting at 10 AM
Bring laptop and charger
```

15. tail

Description: Displays the last part of a file, by default the last 10 lines. The -n option sets a custom line count and -f follows a file live as new lines are appended (useful for log files).

Syntax: `tail [-n number] filename`

Output:

```
isha@Ubuntu:~/Desktop$ tail -n 2 notes.txt
Bring laptop and charger
Submit report by Friday
```

16. history

Description: Displays a numbered list of previously executed commands from the current shell session's command history.

Syntax: `history`

Output:

```
isha@Ubuntu:~/Desktop$ history
  1  pwd
  2  ls -la
  3  mkdir Projects
  4  cd Projects
  5  touch report.txt
```

17. grep

Description: Searches for a specified text pattern inside one or more files and prints the matching lines. Common options: -i (case-insensitive), -r (recursive search), -n (show line numbers).

Syntax: `grep [options] "pattern" filename`

Output:

```
isha@Ubuntu:~/Desktop$ grep "report" notes.txt
Submit report by Friday
```

18. diff

Description: Compares two files line by line and displays the differences between them, showing which lines were added, removed or changed.

Syntax: `diff file1 file2`

Output:

```
isha@Ubuntu:~/Desktop$ diff notes.txt notes_backup.txt
3c3
< Submit report by Friday
---
```

```
> Submit report by Monday
```

19. vi

Description: Opens the Vi text editor to create a new file or edit an existing one directly in the terminal. It operates in modes: Normal mode for navigation/commands, Insert mode (press i) for typing text and Command-line mode (press :) for saving/quitting (e.g., :wq to save and exit, :q! to quit without saving).

Syntax: `vi filename`

Output:

```
isha@Ubuntu:~/Desktop$ vi notes.txt
```

Note: *(Opens notes.txt in the Vi editor interface within the terminal for editing)*

20. sed

Description: A stream editor used to search, replace, delete and modify text in files or input streams, typically without opening the file in an editor. The 's' command performs substitution.

Syntax: `sed 's/old_text/new_text/' filename`

Output:

```
isha@Ubuntu:~/Desktop$ sed 's/Friday/Monday/' notes.txt
Meeting at 10 AM
Bring laptop and charger
Submit report by Monday
```

21. apt

Description: The package management tool used on Debian and Ubuntu-based systems to install, update, upgrade and remove software packages.

Syntax: `sudo apt install package_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo apt install curl
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
  curl
0 upgraded, 1 newly installed, 0 to remove.
Setting up curl ...
curl installed successfully
```

22. yum

Description: The package management tool used on older RHEL/CentOS-based systems to install, update and remove RPM packages.

Syntax: `sudo yum install package_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo yum install httpd
Loaded plugins: fastestmirror
Resolving Dependencies
```

```
--> Running transaction check
Installing : httpd
Complete!
```

23. dnf

Description: The modern package manager for RHEL/Fedora-based systems, serving as the successor to yum with improved dependency resolution and performance.

Syntax: `sudo dnf install package_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo dnf install httpd
Last metadata expiration check completed.
Dependencies resolved.
Installing: httpd
Complete!
```

24. sudo

Description: Executes a command with elevated (superuser/root) privileges. It prompts for the current user's password before running the command, provided the user has sudo permissions.

Syntax: `sudo command`

Output:

```
isha@Ubuntu:~/Desktop$ sudo apt update
[sudo] password for user:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Reading package lists... Done
```

25. wget

Description: Downloads files from the internet over HTTP, HTTPS or FTP directly from the command line, saving them to the current directory.

Syntax: `wget URL`

Output:

```
isha@Ubuntu:~/Desktop$ wget https://example.com/file.zip
Resolving example.com... 93.184.216.34
Connecting to example.com|93.184.216.34|:443... connected.
HTTP request sent, awaiting response... 200 OK
Saving to: 'file.zip'
file.zip           100%[==================>]  1.2M  --.-KB/s    in 0.4s
'file.zip' saved
```

26. curl

Description: Transfers data to or from a server using various protocols (HTTP, HTTPS, FTP, etc.). Commonly used to make HTTP requests, download files or test APIs. The -O option saves the output with the remote file's name.

Syntax: `curl [options] URL`

Output:

```
isha@Ubuntu:~/Desktop$ curl https://api.example.com/status
{"status": "ok", "uptime": "12h 45m"}
```

27. ping

Description: Tests network connectivity to a host by sending ICMP echo request packets and measuring the response time. Press Ctrl+C to stop.

Syntax: `ping hostname_or_IP`

Output:

```
isha@Ubuntu:~/Desktop$ ping google.com
PING google.com (142.250.premium): 56 data bytes
64 bytes from 142.250.x.x: icmp_seq=0 ttl=115 time=12.4 ms
64 bytes from 142.250.x.x: icmp_seq=1 ttl=115 time=11.8 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

28. free

Description: Displays the total, used and available amount of physical (RAM) and swap memory on the system. The -h option shows values in human-readable units.

Syntax: `free [-h]`

Output:

```
isha@Ubuntu:~/Desktop$ free -h
total    used    free   shared  buff/cache  available
Mem:  7.8Gi   2.1Gi   3.2Gi   220Mi   2.5Gi        5.3Gi
Swap: 2.0Gi      0B   2.0Gi
```

29. top

Description: Displays a real-time, continuously updating view of running processes and system resource usage (CPU, memory). Press 'q' to quit.

Syntax: `top`

Output:

```
isha@Ubuntu:~/Desktop$ top
top - 11:42:10 up 2 days,  3:15,  1 user,  load average: 0.15, 0.10, 0.08
Tasks: 210 total,   1 running, 209 sleeping
%Cpu(s):  3.2 us,  1.1 sy,  0.0 ni, 95.4 id
MiB Mem :   7987.0 total,   3312.4 free
  PID USER   %CPU  %MEM   COMMAND
 1024 user    2.3   1.5   firefox
  512 root    0.5   0.3   systemd
```

30. zip

Description: Compresses one or more files or directories into a single .zip archive file. The -r option is used to include directories recursively.

Syntax: `zip [options] archive_name.zip file_or_folder`

Output:

```
isha@Ubuntu:~/Desktop$ zip -r archive.zip Documents/
  adding: Documents/ (stored 0%)
  adding: Documents/notes.txt (deflated 32%)
  adding: Documents/report.txt (deflated 28%)
```

31. unzip

Description: Extracts the contents of a .zip archive file into the current directory (or a specified destination).

Syntax: `unzip archive_name.zip`

Output:

```
isha@Ubuntu:~/Desktop$ unzip archive.zip
Archive:  archive.zip
   creating: Documents/
  inflating: Documents/notes.txt
  inflating: Documents/report.txt
```

32. find

Description: Searches for files and directories within a directory hierarchy based on criteria such as name, type, size or modification time.

Syntax: `find path -name "filename"`

Output:

```
isha@Ubuntu:~/Desktop$ find . -name "*.txt"
./notes.txt
./Documents/report.txt
./Documents/notes_backup.txt
```

33. locate

Description: Quickly searches for files by name using a pre-built index database, making it much faster than find, though results depend on how recently the database was updated.

Syntax: `locate filename`

Output:

```
isha@Ubuntu:~/Desktop$ locate notes.txt
/home/user/notes.txt
/home/user/Documents/notes_backup.txt
```

34. updatedb

Description: Updates the database used by the locate command so that recently created or moved files can be found. Typically requires root privileges and is run periodically or manually.

Syntax: `sudo updatedb`

Output:

```
isha@Ubuntu:~/Desktop$ sudo updatedb
```

Note: *(No output on success; the locate database is refreshed in the background)*

35. useradd

Description: Creates a new user account on the system. Typically requires root/sudo privileges. The -m option creates a home directory for the new user.

Syntax: `sudo useradd [options] username`

Output:

```
isha@Ubuntu:~/Desktop$ sudo useradd -m john
```

Note: *(No output on success; a new user 'john' and home directory /home/john are created)*

36. passwd

Description: Sets or changes the password for a user account. Running it without an argument changes the current user's own password.

Syntax: `sudo passwd username`

Output:

```
isha@Ubuntu:~/Desktop$ sudo passwd john
New password:
Retype new password:
passwd: password updated successfully
```

37. groupadd

Description: Creates a new user group on the system, which can then be used to organize permissions for multiple users at once.

Syntax: `sudo groupadd group_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo groupadd developers
```

Note: *(No output on success; the 'developers' group is created)*

38. usermod

Description: Modifies an existing user account's properties, such as adding the user to a group (-aG), changing the home directory (-d) or changing the login shell (-s).

Syntax: `sudo usermod [options] username`

Output:

```
isha@Ubuntu:~/Desktop$ sudo usermod -aG developers john
```

Note: *(No output on success; user 'john' is added to the 'developers' group)*

39. chmod

Description: Changes the read, write and execute permissions of a file or directory, using either symbolic notation (u/g/o + r/w/x) or numeric (octal) notation.

Syntax: `chmod [permissions] filename`

Output:

```
isha@Ubuntu:~/Desktop$ chmod 755 script.sh
isha@Ubuntu:~/Desktop$ ls -l script.sh
```

```
-rwxr-xr-x 1 user user 128 Sep  3 11:20 script.sh
```

40. file

Description: Determines and displays the type of a file (e.g. text, directory, executable, image) by examining its content rather than relying on the file extension.

Syntax: `file filename`

Output:

```
isha@Ubuntu:~/Desktop$ file notes.txt
notes.txt: ASCII text
```

41. stat

Description: Displays detailed status information about a file or directory, including size, permissions, owner and timestamps.

Syntax: `stat filename`

Output:

```
isha@Ubuntu:~/Desktop$ stat notes.txt
  File: notes.txt
  Size: 45          Blocks: 8    regular file
Access: (0644/-rw-r--r--)  Uid: (1000/isha)
Modify: 2026-09-01 10:12:03
```

42. tree

Description: Displays the contents of a directory as an indented tree structure, showing subdirectories and files at a glance. May need to be installed (sudo apt install tree).

Syntax: `tree [directory]`

Output:

```
isha@Ubuntu:~/Desktop$ tree
.
├── Documents
│   └── notes.txt
├── Downloads
└── Pictures

2 directories, 1 file
```

43. less

Description: Opens a file for page-by-page viewing in the terminal, allowing scrolling both forward and backward. Press q to quit.

Syntax: `less filename`

Output:

```
isha@Ubuntu:~/Desktop$ less notes.txt
```

Note: *(Opens notes.txt in a scrollable pager; use arrow keys to navigate, q to exit)*

44. more

Description: Displays a file one screen at a time, similar to less but with simpler, mostly forward-only navigation. Press space to scroll, q to quit.

Syntax: `more filename`

Output:

```
isha@Ubuntu:~/Desktop$ more notes.txt
```

Note: *(Displays notes.txt one page at a time; press space for the next page, q to quit)*

45. nano

Description: Opens the Nano text editor, a simple and beginner-friendly terminal editor. Use Ctrl+O to save and Ctrl+X to exit.

Syntax: `nano filename`

Output:

```
isha@Ubuntu:~/Desktop$ nano notes.txt
```

Note: *(Opens notes.txt in the Nano editor interface within the terminal for editing)*

46. vim

Description: Opens Vim, an advanced and highly configurable version of Vi with extra features like syntax highlighting and plugins.

Syntax: `vim filename`

Output:

```
isha@Ubuntu:~/Desktop$ vim notes.txt
```

Note: *(Opens notes.txt in the Vim editor interface within the terminal for editing)*

47. wc

Description: Counts and displays the number of lines, words and characters in a file. Options: -l (lines only), -w (words only), -c (characters only).

Syntax: `wc [options] filename`

Output:

```
isha@Ubuntu:~/Desktop$ wc notes.txt
 3  15 45 notes.txt
```

48. sort

Description: Sorts the lines of a file alphabetically or numerically and prints the result. -r reverses the order and -n sorts numerically.

Syntax: `sort filename`

Output:

```
isha@Ubuntu:~/Desktop$ sort names.txt
Alice
Bob
Charlie
```

49. uniq

Description: Filters out adjacent duplicate lines from a file or input, leaving only unique lines. Input is typically piped from sort first.

Syntax: `sort filename | uniq`

Output:

```
isha@Ubuntu:~/Desktop$ sort fruits.txt | uniq
apple
banana
mango
```

50. which

Description: Shows the full path of the executable that would run for a given command, based on directories listed in the PATH variable.

Syntax: `which command_name`

Output:

```
isha@Ubuntu:~/Desktop$ which python3
/usr/bin/python3
```

51. whereis

Description: Locates the binary, source and manual page files for a command, searching standard system directories.

Syntax: `whereis command_name`

Output:

```
isha@Ubuntu:~/Desktop$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz
```

52. xargs

Description: Builds and executes commands using items read from standard input, commonly used to pass the output of one command as arguments to another.

Syntax: `command | xargs another_command`

Output:

```
isha@Ubuntu:~/Desktop$ find . -name "*.txt" | xargs rm
```

Note: *(No output on success; every .txt file found is deleted)*

53. chown

Description: Changes the owner (and optionally the group) of a file or directory. Requires sudo when changing ownership of files you don't own.

Syntax: `sudo chown user:group filename`

Output:

```
isha@Ubuntu:~/Desktop$ sudo chown isha:isha file.txt
```

Note: *(No output on success; the owner and group of file.txt are updated)*

54. chgrp

Description: Changes the group ownership of a file or directory without affecting the owner.

Syntax: `sudo chgrp group_name filename`

Output:

```
isha@Ubuntu:~/Desktop$ sudo chgrp developers file.txt
```

Note: *(No output on success; the group of file.txt is changed to 'developers')*

55. umask

Description: Displays or sets the default permission mask applied to newly created files and directories.

Syntax: `umask [value]`

Output:

```
isha@Ubuntu:~/Desktop$ umask
0022
```

56. id

Description: Displays the user ID (UID), group ID (GID) and group memberships of the current or specified user.

Syntax: `id [username]`

Output:

```
isha@Ubuntu:~/Desktop$ id
uid=1000(isha) gid=1000(isha) groups=1000(isha),27(sudo)
```

57. whoami

Description: Prints the username of the currently logged-in user.

Syntax: `whoami`

Output:

```
isha@Ubuntu:~/Desktop$ whoami
isha
```

58. groups

Description: Displays the groups that the current or specified user belongs to.

Syntax: `groups [username]`

Output:

```
isha@Ubuntu:~/Desktop$ groups
isha sudo developers
```

59. su

Description: Switches to another user account (commonly root), prompting for that user's password. Without a username, it switches to root.

Syntax: `su [username]`

Output:

```
isha@Ubuntu:~/Desktop$ su -
Password:
root@Ubuntu:~#
```

60. adduser

Description: A user-friendly, interactive front-end to useradd used on Debian/Ubuntu systems to create a new user account, prompting for a password and details.

Syntax: `sudo adduser username`

Output:

```
isha@Ubuntu:~/Desktop$ sudo adduser john
Adding user 'john' ...
Enter new UNIX password:
Retype new UNIX password:
Adding new user 'john' to the system.
```

61. userdel

Description: Deletes an existing user account from the system. The -r option also removes the user's home directory and mail spool.

Syntax: `sudo userdel [options] username`

Output:

```
isha@Ubuntu:~/Desktop$ sudo userdel -r john
```

Note: *(No output on success; user 'john' and their home directory are removed)*

62. groupdel

Description: Deletes an existing group from the system, provided no user has it set as their primary group.

Syntax: `sudo groupdel group_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo groupdel developers
```

Note: *(No output on success; the 'developers' group is removed)*

63. gpasswd

Description: Administers the /etc/group file, used to add or remove users from a group or manage a group's administrators.

Syntax: `sudo gpasswd -a username group_name`

Output:

```
isha@Ubuntu:~/Desktop$ sudo gpasswd -a john developers
Adding user john to group developers
```

64. getent

Description: Retrieves entries from administrative databases such as passwd or group, whether they come from local files or a network source like LDAP.

Syntax: `getent database [key]`

Output:

```
isha@Ubuntu:~/Desktop$ getent passwd isha
isha:x:1000:1000:isha,,,:/home/isha:/bin/bash
```

65. ps

Description: Displays information about currently running processes for the current shell session by default.

Syntax: `ps [options]`

Output:

```
isha@Ubuntu:~/Desktop$ ps
  PID TTY          TIME CMD
 2145 pts/0    00:00:00 bash
 2210 pts/0    00:00:00 ps
```

66. ps aux

Description: Displays a detailed list of all running processes on the system, from every user, including their CPU and memory usage.

Syntax: `ps aux`

Output:

```
isha@Ubuntu:~/Desktop$ ps aux
USER   PID %CPU %MEM COMMAND
root     1  0.0  0.1 /sbin/init
isha  2145  0.1  0.3 bash
isha  3312  1.2  2.1 firefox
```

67. htop

Description: An interactive, color-coded process viewer similar to top but easier to navigate. May need to be installed (sudo apt install htop).

Syntax: `htop`

Output:

```
isha@Ubuntu:~/Desktop$ htop
```

Note: *(Opens an interactive, full-screen process viewer; press F10 or q to quit)*

68. pgrep

Description: Searches for running processes matching a given name and prints their process IDs (PIDs).

Syntax: `pgrep process_name`

Output:

```
isha@Ubuntu:~/Desktop$ pgrep firefox
3312
```

69. kill

Description: Sends a signal (SIGTERM by default) to a process to request it stop, using its process ID (PID).

Syntax: `kill [signal] PID`

Output:

```
isha@Ubuntu:~/Desktop$ kill 3312
```

Note: *(No output on success; the process with PID 3312 is terminated)*

70. killall

Description: Terminates all running processes that match a given process name, rather than requiring a specific PID.

Syntax: `killall process_name`

Output:

```
isha@Ubuntu:~/Desktop$ killall firefox
```

Note: *(No output on success; all running firefox processes are terminated)*

71. pkill

Description: Sends a signal to processes matching a name or other attribute pattern, similar to a combination of pgrep and kill.

Syntax: `pkill process_name`

Output:

```
isha@Ubuntu:~/Desktop$ pkill firefox
```

Note: *(No output on success; matching firefox processes are terminated)*

72. jobs

Description: Lists the background and stopped jobs running in the current shell session, along with their job numbers and status.

Syntax: `jobs`

Output:

```
isha@Ubuntu:~/Desktop$ jobs
[1]+  Running    sleep 100 &
```

73. bg

Description: Resumes a suspended job and runs it in the background, freeing up the terminal for other commands.

Syntax: `bg [job_spec]`

Output:

```
isha@Ubuntu:~/Desktop$ bg %1
[1]+ sleep 100 &
```

74. fg

Description: Brings a background or suspended job back to the foreground, giving it control of the terminal again.

Syntax: `fg [job_spec]`

Output:

```
isha@Ubuntu:~/Desktop$ fg %1
sleep 100
```

75. nohup

Description: Runs a command so it keeps executing even after the terminal session that started it is closed, ignoring the hangup signal.

Syntax: `nohup command &`

Output:

```
isha@Ubuntu:~/Desktop$ nohup backup.sh &
[1] 4021
nohup: ignoring input and appending output to 'nohup.out'
```

76. uname

Description: Prints system information such as the kernel name, version and hardware architecture. The -a option shows all available information.

Syntax: `uname [-a]`

Output:

```
isha@Ubuntu:~/Desktop$ uname -a
Linux Ubuntu 6.8.0-45-generic #45-Ubuntu SMP x86_64 GNU/Linux
```

77. hostname

Description: Displays or temporarily sets the system's hostname (the name that identifies the machine on a network).

Syntax: `hostname`

Output:

```
isha@Ubuntu:~/Desktop$ hostname
Ubuntu
```

78. hostnamectl

Description: Displays or changes the system hostname and related system details; the modern replacement for plain hostname on systemd-based systems.

Syntax: `hostnamectl`

Output:

```
isha@Ubuntu:~/Desktop$ hostnamectl
Static hostname: Ubuntu
Operating System: Ubuntu 24.04 LTS
Kernel: Linux 6.8.0-45-generic
```

79. uptime

Description: Shows how long the system has been running, along with the number of logged-in users and system load averages.

Syntax: `uptime`

Output:

```
isha@Ubuntu:~/Desktop$ uptime
 11:42:10 up 2 days,  3:15,  1 user,  load average: 0.15, 0.10, 0.08
```

80. date

Description: Displays or sets the current system date and time. Supports custom output formats using format specifiers.

Syntax: `date [+format]`

Output:

```
isha@Ubuntu:~/Desktop$ date
Mon Sep  7 11:42:10 IST 2026
```

81. lscpu

Description: Displays detailed information about the CPU architecture, including the number of cores, threads and clock speed.

Syntax: `lscpu`

Output:

```
isha@Ubuntu:~/Desktop$ lscpu
Architecture:  x86_64
CPU(s):        8
Model name:    Intel(R) Core(TM) i5-10300H
```

82. lsblk

Description: Lists all block devices (disks and partitions) attached to the system in a tree-like format.

Syntax: `lsblk`

Output:

```
isha@Ubuntu:~/Desktop$ lsblk
NAME   SIZE RO TYPE MOUNTPOINT
sda    100G  0 disk
├─sda1  99G  0 part /
└─sda2   1G  0 part /boot
```

83. lsusb

Description: Lists all USB devices currently connected to the system, along with their vendor and product IDs.

Syntax: `lsusb`

Output:

```
isha@Ubuntu:~/Desktop$ lsusb
Bus 001 Device 002: ID 8087:0026 Intel Corp.
Bus 002 Device 003: ID 046d:c52b Logitech Mouse
```

84. lspci

Description: Lists all PCI devices connected to the system, such as the graphics card, network adapter and USB controllers.

Syntax: `lspci`

Output:

```
isha@Ubuntu:~/Desktop$ lspci
00:00.0 Host bridge: Intel Corporation Device
00:02.0 VGA controller: Intel UHD Graphics
```

85. df

Description: Reports the amount of disk space used and available on mounted filesystems. The -h option shows sizes in human-readable form.

Syntax: `df [-h]`

Output:

```
isha@Ubuntu:~/Desktop$ df -h
Filesystem   Size  Used Avail Use% Mounted on
/dev/sda1     99G   42G   52G  45% /
```

86. du

Description: Estimates and displays the disk space used by files and directories. -h shows human-readable sizes, -s summarizes a total.

Syntax: `du [-sh] path`

Output:

```
isha@Ubuntu:~/Desktop$ du -sh Documents/
128M    Documents/
```

87. dmesg

Description: Displays messages from the kernel ring buffer, useful for viewing boot messages and hardware-related events.

Syntax: `dmesg`

Output:

```
isha@Ubuntu:~/Desktop$ dmesg | tail -n 3
[    5.812345] usb 1-1: new high-speed USB device
[    5.912345] input: Logitech USB Mouse
[   12.203456] Wi-Fi: Link Up
```

88. mount

Description: Attaches a filesystem (such as a USB drive or disk partition) to a specified directory (mount point) so it can be accessed.

Syntax: `sudo mount /dev/device /mount/point`

Output:

```
isha@Ubuntu:~/Desktop$ sudo mount /dev/sdb1 /mnt/usb
```

Note: *(No output on success; the device is mounted at /mnt/usb)*

89. umount

Description: Detaches (unmounts) a previously mounted filesystem, making it safe to remove the device.

Syntax: `sudo umount /mount/point`

Output:

```
isha@Ubuntu:~/Desktop$ sudo umount /mnt/usb
```

Note: *(No output on success; the filesystem is safely unmounted)*

90. fdisk

Description: A menu-driven utility used to view or modify a disk's partition table. Requires root privileges.

Syntax: `sudo fdisk -l`

Output:

```
isha@Ubuntu:~/Desktop$ sudo fdisk -l
Disk /dev/sda: 100 GiB
Device     Boot  Start       End Size Type
/dev/sda1  *      2048 207618047  99G Linux
```

91. blkid

Description: Displays the UUID, filesystem type and label of block devices, useful for identifying disks in files like /etc/fstab.

Syntax: `sudo blkid`

Output:

```
isha@Ubuntu:~/Desktop$ sudo blkid
/dev/sda1: UUID="a1b2c3d4-..." TYPE="ext4"
```

92. findmnt

Description: Displays a tree of currently mounted filesystems in a readable, tabular format and can verify entries in /etc/fstab.

Syntax: `findmnt [target]`

Output:

```
isha@Ubuntu:~/Desktop$ findmnt /
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/sda1 ext4   rw,relatime
```

93. ip

Description: A versatile networking utility used to show or configure network interfaces, addresses and routes. Common subcommands: ip addr, ip link, ip route.

Syntax: `ip [addr | link | route]`

Output:

```
isha@Ubuntu:~/Desktop$ ip addr
```

```
1: lo: <LOOPBACK,UP> mtu 65536
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,UP> mtu 1500
    inet 192.168.1.10/24 scope global eth0
```

94. ssh

Description: Opens a secure, encrypted remote login session to another machine over the network.

Syntax: `ssh username@hostname`

Output:

```
isha@Ubuntu:~/Desktop$ ssh isha@192.168.1.20
isha@192.168.1.20's password:
Welcome to Ubuntu 24.04 LTS
isha@remote-server:~$
```

95. scp

Description: Securely copies files or directories between the local machine and a remote machine over SSH.

Syntax: `scp source username@host:destination`

Output:

```
isha@Ubuntu:~/Desktop$ scp notes.txt isha@192.168.1.20:/home/isha/
notes.txt          100%   45   1.2KB/s   00:00
```

96. rsync

Description: Efficiently synchronizes files and directories between locations, locally or over a network, copying only the changes.

Syntax: `rsync [options] source destination`

Output:

```
isha@Ubuntu:~/Desktop$ rsync -av Documents/ /mnt/backup/
sending incremental file list
Documents/
Documents/notes.txt

sent 1,204 bytes  received 35 bytes
```

97. ss

Description: Displays detailed information about network socket connections and is the modern replacement for netstat.

Syntax: `ss [options]`

Output:

```
isha@Ubuntu:~/Desktop$ ss -tuln
Netid State  Local Address:Port  Peer Address:Port
tcp   LISTEN 0.0.0.0:22           0.0.0.0:*
```

98. netstat

Description: Displays network connections, routing tables and interface statistics. May need to be installed separately (net-tools package).

Syntax: `netstat [options]`

Output:

```
isha@Ubuntu:~/Desktop$ netstat -tuln
Proto Local Address     Foreign Address   State
tcp   0.0.0.0:22        0.0.0.0:*         LISTEN
```

99. traceroute

Description: Traces and displays the path (hops) that packets take to reach a destination host across the network.

Syntax: `traceroute hostname`

Output:

```
isha@Ubuntu:~/Desktop$ traceroute google.com
 1  192.168.1.1  1.2 ms
 2  10.10.0.1    5.4 ms
 3  142.250.1.1 12.1 ms
```

100. dig

Description: Queries DNS servers for information about a domain, such as its IP address, mail servers or name servers.

Syntax: `dig domain_name`

Output:

```
isha@Ubuntu:~/Desktop$ dig google.com
;; ANSWER SECTION:
google.com.  300  IN  A  142.250.195.14
;; Query time: 24 msec
```

101. nslookup

Description: Queries DNS servers to resolve a domain name to its IP address or vice versa.

Syntax: `nslookup domain_name`

Output:

```
isha@Ubuntu:~/Desktop$ nslookup google.com
Server:  127.0.0.53

Non-authoritative answer:
Name: google.com
Address: 142.250.195.14
```

102. host

Description: A simple utility for performing DNS lookups, resolving domain names to IP addresses and vice versa.

Syntax: `host domain_name`

Output:

```
isha@Ubuntu:~/Desktop$ host google.com
google.com has address 142.250.195.14
```

103. nmcli

Description: A command-line tool for controlling NetworkManager, used to view, connect and configure network connections such as Wi-Fi and Ethernet.

Syntax: `nmcli [options]`

Output:

```
isha@Ubuntu:~/Desktop$ nmcli device status
DEVICE  TYPE      STATE        CONNECTION
eth0    ethernet  connected    Wired connection 1
wlo1    wifi      disconnected --
```

104. ufw

Description: Uncomplicated Firewall (UFW) is a simplified command-line interface for managing firewall rules on Ubuntu/Debian systems. Common subcommands: status, enable, disable, allow, deny, delete.

Syntax: `sudo ufw [status | enable | disable | allow port | deny port]`

Output:

```
isha@Ubuntu:~/Desktop$ sudo ufw allow 22
Rule added
Rule added (v6)
```

105. ssh-keygen

Description: Generates a new SSH public/private key pair, used for passwordless and more secure SSH authentication.

Syntax: `ssh-keygen [-t type]`

Output:

```
isha@Ubuntu:~/Desktop$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/isha/.ssh/id_rsa):
Your identification has been saved in id_rsa
```

106. ssh-copy-id

Description: Copies a local public SSH key to a remote server's authorized_keys file, enabling passwordless SSH login.

Syntax: `ssh-copy-id username@hostname`

Output:

```
isha@Ubuntu:~/Desktop$ ssh-copy-id isha@192.168.1.20
Number of key(s) added: 1
```

```
Now try logging into the machine with: "ssh isha@192.168.1.20"
```

107. sftp

Description: Opens an interactive secure file transfer session over SSH, allowing files to be uploaded, downloaded and browsed remotely.

Syntax: `sftp username@hostname`

Output:

```
isha@Ubuntu:~/Desktop$ sftp isha@192.168.1.20
Connected to 192.168.1.20.
sftp>
```

108. openssl

Description: A toolkit for working with SSL/TLS encryption, certificates and general cryptographic operations such as hashing and key generation.

Syntax: `openssl command [options]`

Output:

```
isha@Ubuntu:~/Desktop$ openssl req -x509 -newkey rsa:2048 -keyout key.pem -out
cert.pem
Generating a RSA private key
..........+++++
writing new private key to 'key.pem'
```

109. gpg

Description: Encrypts, decrypts and digitally signs files or messages using the GNU Privacy Guard implementation of OpenPGP.

Syntax: `gpg [options] filename`

Output:

```
isha@Ubuntu:~/Desktop$ gpg -c secret.txt
```

Note: *(Prompts for a passphrase and creates an encrypted secret.txt.gpg file)*

110. last

Description: Displays a list of the most recent user logins, including login time, duration and terminal, read from the login log.

Syntax: `last`

Output:

```
isha@Ubuntu:~/Desktop$ last
isha  pts/0  192.168.1.5  Mon Sep  7 09:12   still logged in
isha  pts/0  192.168.1.5  Sun Sep  6 18:03 - 20:45
```

111. who

Description: Displays a list of users currently logged into the system, along with their terminal and login time.

Syntax: who

Output:

```
isha@Ubuntu:~/Desktop$ who
isha   pts/0   2026-09-07 09:12 (192.168.1.5)
```

112. w

Description: Displays currently logged-in users along with what they are doing, plus system load information (a more detailed version of who).

Syntax: w

Output:

```
isha@Ubuntu:~/Desktop$ w
 11:42:10 up 2 days,  1 user,  load average: 0.15, 0.10, 0.08
USER  TTY   LOGIN@  IDLE  WHAT
isha  pts/0 09:12   0.00s w
```

113. journalctl

Description: Queries and displays logs collected by systemd's journal, covering the kernel, services and boot process. -u filters by unit and -f follows logs live.

Syntax: journalctl [options]

Output:

```
isha@Ubuntu:~/Desktop$ journalctl -u nginx
Sep 07 09:00:01 Ubuntu systemd[1]: Started A high performance web server.
Sep 07 09:00:02 Ubuntu nginx[1200]: Configuration file test successful
```

114. systemctl

Description: Controls systemd services (units) on the system. Common subcommands: status, start, stop, restart, reload, enable, disable, is-active and is-enabled.

Syntax: sudo systemctl [status | start | stop | restart | enable] service

Output:

```
isha@Ubuntu:~/Desktop$ sudo systemctl restart nginx
```

Note: *(No output on success; the nginx service is restarted)*

115. | (pipe)

Description: Sends the standard output of one command directly into the standard input of the next command, allowing commands to be chained together.

Syntax: command1 | command2

Output:

```
isha@Ubuntu:~/Desktop$ ps aux | grep nginx
isha  2233  0.1  0.4 nginx: worker process
```

116. > (overwrite)

Description: Redirects a command's standard output to a file, overwriting the file's existing contents (or creating it if it doesn't exist).

Syntax: `command > filename`

Output:

```
isha@Ubuntu:~/Desktop$ echo "hello linux" > file1.txt
```

Note: *(No output on success; file1.txt now contains 'hello linux', replacing any previous content)*

117. >> (append)

Description: Redirects a command's standard output to a file, appending it to the end of the file's existing contents instead of overwriting them.

Syntax: `command >> filename`

Output:

```
isha@Ubuntu:~/Desktop$ echo "second line" >> file1.txt
```

Note: *(No output on success; 'second line' is added to the end of file1.txt)*

118. < (input)

Description: Redirects the contents of a file to be used as standard input for a command, instead of typing input manually.

Syntax: `command < filename`

Output:

```
isha@Ubuntu:~/Desktop$ sort < names.txt
Alice
Bob
Charlie
```

119. 2> (redirect errors)

Description: Redirects a command's standard error (error messages) to a file, separately from its normal output.

Syntax: `command 2> error_log.txt`

Output:

```
isha@Ubuntu:~/Desktop$ ls /nofolder 2> error_log.txt
```

Note: *(No output on screen; the 'No such file or directory' error is written to error_log.txt)*

120. &> (redirect both)

Description: Redirects both standard output and standard error to the same file.

Syntax: `command &> all_output.txt`

Output:

```
isha@Ubuntu:~/Desktop$ ./backup.sh &> all_output.txt
```

Note: *(No output on screen; both normal output and any errors are written to all_output.txt)*

121. && (run if success)

Description: Runs the next command only if the previous command completed successfully (returned an exit status of 0).

Syntax: `command1 && command2`

Output:

```
isha@Ubuntu:~/Desktop$ mkdir backup && cd backup
```

Note: *(No output on success; the backup directory is created and then entered only if mkdir succeeded)*

122. ; (run sequentially)

Description: Runs multiple commands one after another in sequence, regardless of whether the earlier commands succeeded or failed.

Syntax: `command1 ; command2`

Output:

```
isha@Ubuntu:~/Desktop$ pwd ; ls
/home/isha/Desktop
Documents  Downloads  notes.txt  Pictures
```

123. bash

Description: Starts a new Bash (Bourne Again SHell) session or runs a shell script using the Bash interpreter.

Syntax: `bash filename.sh`

Output:

```
isha@Ubuntu:~/Desktop$ bash backup.sh
Backup completed successfully
```

124. sh

Description: Starts the default system shell (often a lighter POSIX-compliant shell such as dash) or runs a script using it.

Syntax: `sh filename.sh`

Output:

```
isha@Ubuntu:~/Desktop$ sh backup.sh
Backup completed successfully
```

125. echo

Description: Prints text or the value of a variable to the terminal. Commonly used for displaying messages or writing content into files with redirection.

Syntax: `echo "text"`

Output:

```
isha@Ubuntu:~/Desktop$ echo "Hello Linux"
Hello Linux
```

126. read

Description: Reads a line of input from the user and stores it in a variable, commonly used inside shell scripts.

Syntax: `read variable_name`

Output:

```
isha@Ubuntu:~/Desktop$ read name
isha@Ubuntu:~/Desktop$ echo $name
Isha
```

127. alias

Description: Creates a shortcut for a longer command, valid for the current shell session unless saved in a startup file such as .bashrc.

Syntax: `alias name='command'`

Output:

```
isha@Ubuntu:~/Desktop$ alias ll='ls -la'
isha@Ubuntu:~/Desktop$ ll
Documents  Downloads  notes.txt  Pictures
```

128. unalias

Description: Removes a previously defined alias from the current shell session.

Syntax: `unalias name`

Output:

```
isha@Ubuntu:~/Desktop$ unalias ll
```

Note: *(No output on success; the ll alias is removed and is no longer available in this session)*

129. source

Description: Executes a shell script or file within the current shell session so any variables or functions it defines remain available afterward. Also usable via the dot operator.

Syntax: `source filename`

Output:

```
isha@Ubuntu:~/Desktop$ source ~/.bashrc
```

Note: *(No output on success; the current session immediately picks up whatever the file defines or changes)*

130. exit

Description: Ends the current shell session or terminal window. Can optionally be given a numeric exit status.

Syntax: `exit [status]`

Output:

```
isha@Ubuntu:~/Desktop$ exit
```

Note: *(No output; the terminal session or SSH connection is closed)*

131. vmstat

Description: Reports a snapshot of virtual memory, processes, CPU activity and I/O statistics, useful for diagnosing performance issues.

Syntax: `vmstat [interval] [count]`

Output:

```
isha@Ubuntu:~/Desktop$ vmstat 1 3
procs -----memory---- ---swap-- -----io---- -----cpu-----
 r  b   free   buff  cache   si   so   bi   bo us sy id wa
 1  0 524288  10240 210432    0    0    2    5  3  1 95  1
```

132. iostat

Description: Reports CPU utilization and input/output statistics for disks and partitions, useful for spotting disk bottlenecks. May need to be installed (sysstat package).

Syntax: `iostat [options]`

Output:

```
isha@Ubuntu:~/Desktop$ iostat
avg-cpu:  %user %nice %system %iowait  %idle
          3.20   0.00    1.10    0.50  95.20

Device   tps  kB_read/s  kB_wrtn/s
sda      12.4   150.2      80.5
```

133. sar

Description: Collects and reports historical system activity such as CPU, memory and network usage over time. Part of the sysstat package.

Syntax: `sar [options] [interval] [count]`

Output:

```
isha@Ubuntu:~/Desktop$ sar -u 1 3
11:42:01     CPU   %user  %nice %system %iowait  %idle
11:42:02     all    3.20   0.00    1.10    0.50  95.20
```

134. watch

Description: Repeatedly runs a command at a fixed interval (2 seconds by default) and displays the output full-screen, useful for monitoring changing values.

Syntax: `watch [options] command`

Output:

```
isha@Ubuntu:~/Desktop$ watch -n 2 df -h
```

Note: *(Clears and redraws the terminal with the command's output every 2 seconds; press Ctrl+C to stop)*

135. tar

Description: Bundles multiple files or directories into a single archive file and can also extract them. Commonly combined with compression using -z for gzip.

Syntax: `tar -czvf archive.tar.gz folder/`

Output:

```
isha@Ubuntu:~/Desktop$ tar -czvf backup.tar.gz Documents/
Documents/
Documents/notes.txt
```

```
Documents/report.txt
```

136. gzip

Description: Compresses a file using the gzip algorithm, replacing the original file with a .gz version by default.

Syntax: `gzip filename`

Output:

```
isha@Ubuntu:~/Desktop$ gzip notes.txt
```

Note: *(No output on success; notes.txt is replaced with notes.txt.gz)*

137. gunzip

Description: Decompresses a .gz file created by gzip, restoring the original file and removing the compressed version.

Syntax: `gunzip filename.gz`

Output:

```
isha@Ubuntu:~/Desktop$ gunzip notes.txt.gz
```

Note: *(No output on success; notes.txt.gz is replaced with the original notes.txt)*

138. crontab

Description: Edits or lists the scheduled tasks (cron jobs) for the current user, defined using cron's time-based syntax.

Syntax: `crontab -e | crontab -l`

Output:

```
isha@Ubuntu:~/Desktop$ crontab -l
0 2 * * * /home/isha/backup.sh
30 9 * * 1 /home/isha/weekly_report.sh
```

139. cron

Description: The background service that reads crontab files and runs scheduled jobs at the times they specify. It runs automatically and is not typically invoked directly.

Syntax: `sudo systemctl status cron`

Output:

```
isha@Ubuntu:~/Desktop$ sudo systemctl status cron
cron.service - Regular background program processing daemon
     Active: active (running)
```

140. at

Description: Schedules a command to run once at a specified future time rather than on a recurring schedule like cron.

Syntax: `at time`

Output:

```
isha@Ubuntu:~/Desktop$ at 18:00
```

```
warning: commands will be executed using /bin/sh
at> backup.sh
at> <EOT>
job 3 at Mon Sep  7 18:00:00 2026
```

141. awk

Description: A pattern-scanning and text-processing language, often used to extract and print specific columns or fields from structured text.

Syntax: `awk 'pattern {action}' filename`

Output:

```
isha@Ubuntu:~/Desktop$ awk '{print $1}' names.txt
Alice
Bob
Charlie
```

142. cut

Description: Extracts specific sections (columns or characters) from each line of a file, based on a delimiter or fixed character positions.

Syntax: `cut -d 'delimiter' -f field filename`

Output:

```
isha@Ubuntu:~/Desktop$ cut -d ':' -f 1 /etc/passwd
root
daemon
isha
```

143. tr

Description: Translates or deletes characters from standard input, commonly used to convert case or remove specific characters.

Syntax: `command | tr 'set1' 'set2'`

Output:

```
isha@Ubuntu:~/Desktop$ echo "hello" | tr 'a-z' 'A-Z'
HELLO
```

144. jq

Description: A lightweight command-line processor for JSON, used to filter, extract and format fields from JSON data. May need to be installed (sudo apt install jq).

Syntax: `command | jq 'filter'`

Output:

```
isha@Ubuntu:~/Desktop$ curl -s https://api.example.com/status | jq '.status'
"ok"
```

145. yq

Description: A command-line processor for YAML (and JSON), similar to jq but for YAML files. May need to be installed separately.

Syntax: `yq 'filter' filename.yaml`

Output:

```
isha@Ubuntu:~/Desktop$ yq '.services.web.image' docker-compose.yaml
nginx:latest
```

146. make

Description: Automates the building and compiling of software projects by reading instructions from a Makefile and running only the steps needed.

Syntax: `make [target]`

Output:

```
isha@Ubuntu:~/Desktop$ make
gcc -c main.c -o main.o
gcc main.o -o app
Build complete
```